

Dynamic Memory Allocation (DMA)

Dr. Tanvir Ahmed

Static Vs Dynamic Memory Allocation

- Almost all of the variables you have declared so far in intro to Course is Statically allocated memory
- In general, if you declare a variable or array and give the variable or array a name, they are statically allocated memory.
- It means their allocated spaces are not changing.
- In regard to memory allocation the word static has the following meaning:
- Static:
 - the memory requirements are known at compile time.
 - after a program compiles, we can perfectly predict how much memory will be needed
 - Whether you take input or not, still that memory spaces will be used
 - Statically allocated memory takes space from a memory pool called Stack
 - Any statically allocated variable can only have its memory reserved while the function within which it was declared is running.
 - For example, if you declare an int x in function F1, if function F1 has completed, no memory is reserved to store x anymore.

Example of Static memory allocation

- `int Arr[100];`  an array with static size. The size is known during compile time

```
int data(int a, int b)
{
    int sum = 0, A[10] ;
    for(int i=0; i<10; i++)
    {
        A[i] = a*b*i;
        sum = sum + A[i];
        printf("%d", A[i]);
    }
    return sum;
}
```

- You cannot return this array like a normal int, float, char, double, or struct variables.
- Why??? Because the array will be destroyed after processing this function!!!

//also remember that we cannot return this statically declared array from function.

- Do you think sum will alive after completing the job of the function?
- The above function declares an array in it. If you call the function, after processing the task in the function, the variables a, b, sum, and the array A[] all will be destroyed.
- Only the value of sum will be returned.

In case of static memory allocation, the memory is automatically managed and destroyed

How about variable length array (VLA) in C?

- VLA was introduced in C99
- Consider the following example:

```
int size;  
scanf("%d", &size);  
int Arr[size];
```
- In this example, the size of Arr is not known during compile time
 - It uses a memory pool called stack to allocate this memory
 - The size of stack is limited and there is a high chance of crash if the array size is large
 - There is no option to determine whether it will crash or not
 - **Questionable programming practice and there are many complains on this practice**
 - Similar to other statically allocated memory, the memory for the array will be destroyed (freed) automatically after the scope of this array
 - It means you cannot access this array after the function where you have declared it.

Static vs Dynamic Memory Allocation

- **Dynamic:**
- the memory requirements are NOT known at compile-time.
- Memory allocation size may vary based on different execution as it may depends on input.
- There is **no name** of the allocated memory. We need a pointer to keep the allocated address.
- Dynamically allocated memory isn't "freed" automatically at the end of the function within which it's declared (*although it is not accessible automatically from another function*)
- This shifts the responsibility of freeing the memory to the programmer.
- This can be done with the *free* function.
- Dynamically allocated memory takes space from a memory pool called heap (not stack)
- You can check whether the memory was allocated or failed

What is your opinion?

- Dynamically allocated memory is not freed automatically like statically allocated memory.
 - Is it good or bad?

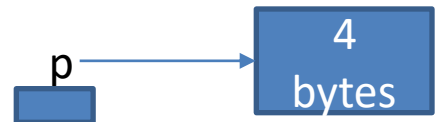
WITH GREAT PERFORMANCE



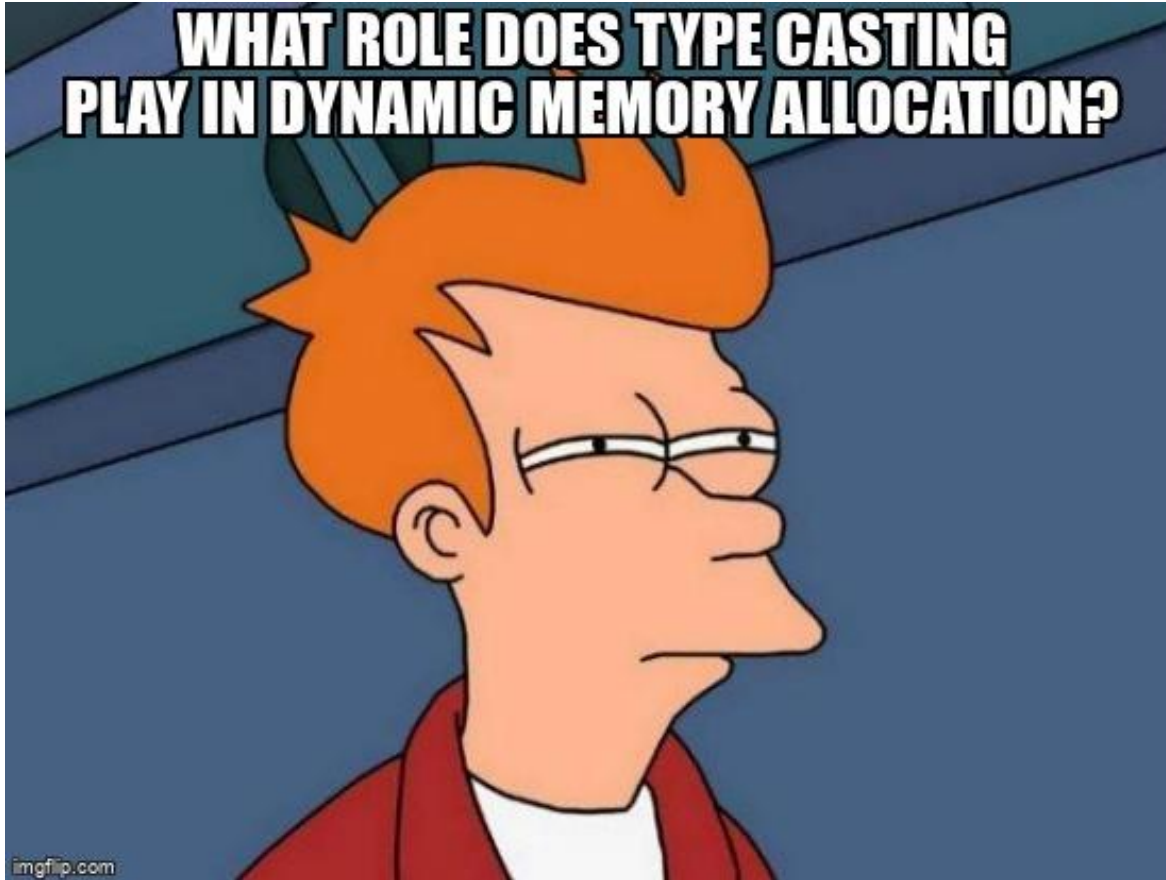
COMES GREAT MEMORY RESPONSIBILITY

malloc, calloc functions

- Malloc or calloc function can be used to dynamically allocate memory
- Malloc function prototype:
 - `void *malloc(size_t size);`
 - It allocates unused space for an object whose *size in bytes is specified by size*
 - The value in the allocated space is unspecified in malloc. So, the space contains garbage value
 - returns a pointer to the beginning of the memory allocated.
 - If the memory can't be found, NULL is returned.
- An example line allocating memory for an int using malloc:
 - `int *p = (int *) malloc(sizeof(int));`
 - *This is happening in the above line:*
 - Allocates 4 bytes of memory (as sizeof(int) is 4 bytes). The memory has garbage values
 - After allocating the memory, malloc returns the address (starting address)
 - It is casted to int pointer so that an int pointer can point to that address
 - Here (int *) line is converting/casting void pointer to int pointer
 - So, now you have a memory for an int!



**WHAT ROLE DOES TYPE CASTING
PLAY IN DYNAMIC MEMORY ALLOCATION?**



malloc, calloc functions

- **Calloc:**

- *void *calloc(size_t nelem, size_t elsize);*
- It allocates an *array of size nelem with each element of size elsize*
- returns a pointer to the beginning of the memory allocated.
- **The spaces shall be initialized to all bits 0**
- If the memory can't be found, NULL is returned.

- *An example line allocating memory for an int using calloc:*

- *int *p = (int *) calloc(1, sizeof(int));*
- ***This is happening in the above line:***
 - *Allocates one 4 bytes of memory (as sizeof(int) is 4 bytes). The memory has 0 as the value in it*
 - *After allocating the memory, calloc returns the address (starting address)*
 - *It is casted to int pointer so that an int pointer can point to that address*
 - *Here (int *) line is converting/casting void pointer to int pointer*
 - *So, now you have a memory for an int!*

- Basically in both function you have to say how many bytes to allocate (how to specify is different).
- Then, if the function successfully finds the memory it returns the pointer to the beginning of the block of the memory returned.
- If unsuccessful, NULL is returned.

Dynamically Allocated Arrays

- Sometimes you don't know how big an array you will need until-runtime.
- You can dynamically allocate memory in those cases.
- `int *ptr1 = (int*) malloc(10*sizeof(int));`
- Remember that `malloc` and `calloc` return **void pointers (void*)**. So, if you want to use the allocated memory as an array, you must **cast** the array to the type you want.
- When you cast it, it knows what type of array it is.
- In some modern compiler, you can avoid casting. In that case it performs the casting automatically based on the type of pointer you are using.
- The above line could be written in multiple lines:

```
int *ptr1;
```

```
ptr1 = (int*) malloc(10*sizeof(int));
```

Now `ptr1` can be treated as an array and you can iterate through it!

-What values are stored in the array?

- `ptr1 = (int*) calloc(10, sizeof(int));`
- How about the values of the `ptr1` now if you use `calloc`?
- See the code in the next slide:



`ptr1` is pointing to the starting address of the allocated memory. So, you can use it like an array!
`ptr1[0], ptr1[1],
....ptr1[9]`

Example malloc and calloc

```
void main()
{
    int *ptr1 = (int*) malloc(10*sizeof
(int)); //why are we casting?
    int *ptr2;
    int i;
    if (ptr1 == NULL)
    {
        printf("Could not allocate
memory\n");
        exit(-1);
    }
    else
    {
        printf("Memory allocated.
Printing data: \n");
        for(i=0; i<10; i++)
            printf("%d ", ptr1[i]);
        //it will print garbage values
    }
    //due to lack of space, the next lines
are shown in the right side
```

```
ptr2 = (int*) calloc(10, sizeof
(int));
    if (ptr2 == NULL)
    {
        printf("Could not allocate
memory\n");
        exit(-1);
    }
    else
    {
        printf("Memory allocated. Printing
data after calloc: \n");
        for(i=0; i<10; i++)
        {
            printf("%d ", ptr2[i]);
        }
        //it will print 0 for all elements
    }
    free(ptr1);
    free(ptr2);
}
```



free the memory

Some notes to remember

- malloc and calloc allocates specified amount of **bytes**
- returns **void***
- So, cast them so that you can iterate through it or let it know what type of data are you going to refer with this.
- Additional notes: In recent compilers you can skip casting, because your program will cast it automatically based on the target pointer type
- Let's say you have a structure called **Student** with *name* and *score*. How would you allocate and create an array of N students?
- *Struct Student *students;*
- *students = (struct Student*) malloc(N*sizeof (struct Student));*

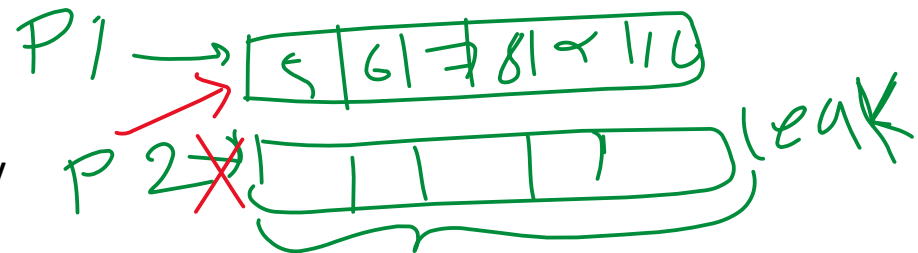
Memory Leak!!!

Can you find an issue in the following code?

```
int* readArray(int size) {  
    int* p1 = (int *)malloc(size*sizeof(int));  
    int* p2 = (int *)malloc(size*sizeof(int));  
    for (i = 0; i<size; i++)  
        scanf("%d", &p1[i]);  
    p2 = p1;  
    return p1;  
}
```

This is going on in the above code:

- P1 is pointing to an allocated array
- P2 is pointing to another allocated array
- P1 being filled up in the loop.
- But after writing P2 = P1, both P1 and P2 is pointing to the allocated array for P1
- The allocated memory for P2 got disconnected!. It is still alive, blocking your memory, but no one can point it anymore!
- It is called memory leak



Memory Leak could really harm system performance!

- Think about you have developed a browser:
 - When you create a new tab, you allocate memory
 - But when you close the tab, you are not freeing the memory
 - That tab is still running the background and taking memory
 - So, if you are running the browser for a while and time to time you will open many tabs and close them, but all of them are taking memory
 - Now you can imagine the performance of the system!
- So, free dynamically allocated memory if you do not use them!



Another Example Step by Step

- `int x;`



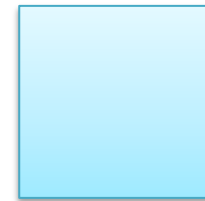
x

Another Example Step by Step

- `int x;`
- `int *p1;`



x

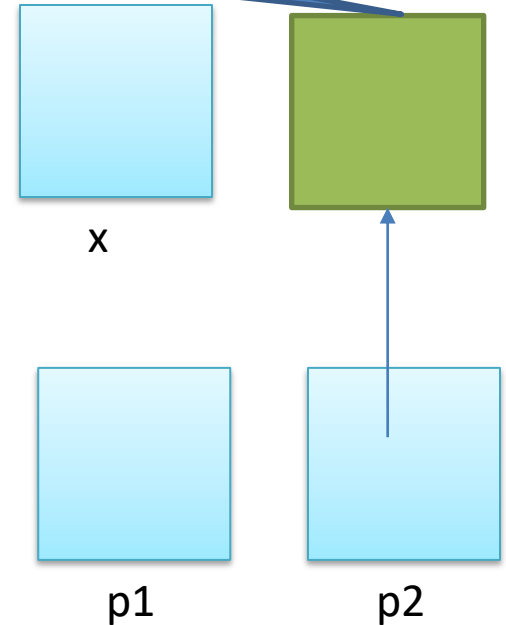


p1

Another Example Step by Step

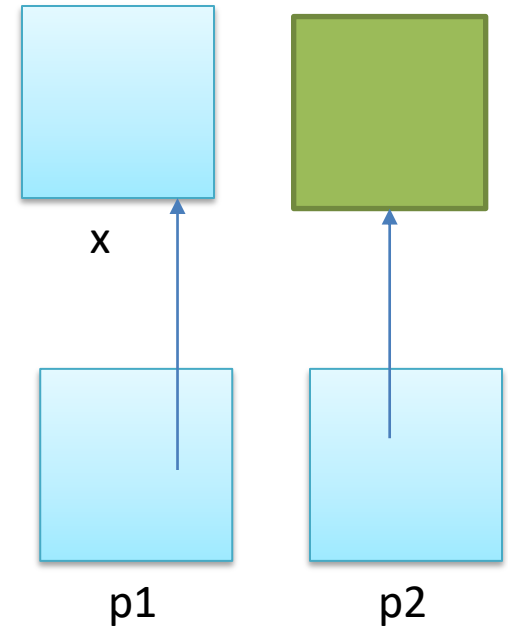
Do not have
any name!

- `int x;`
- `int *p1;`
- `int *p2 = (int *) malloc(sizeof(int));`



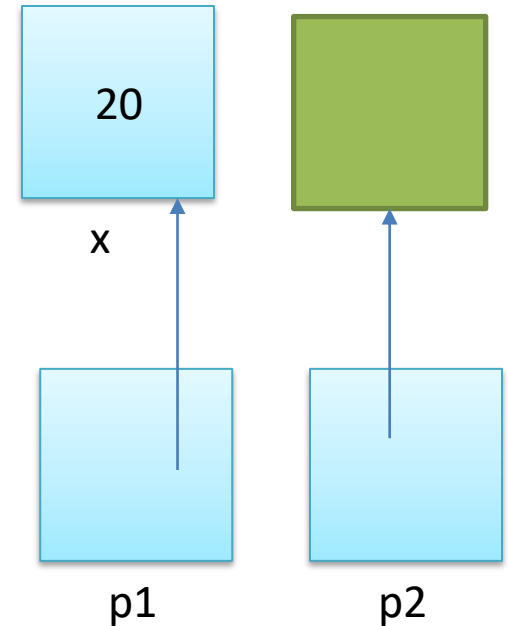
Another Example Step by Step

- `int x;`
- `int *p1;`
- `int *p2 = (int *) malloc(sizeof(int));`
- `p1 = &x;`



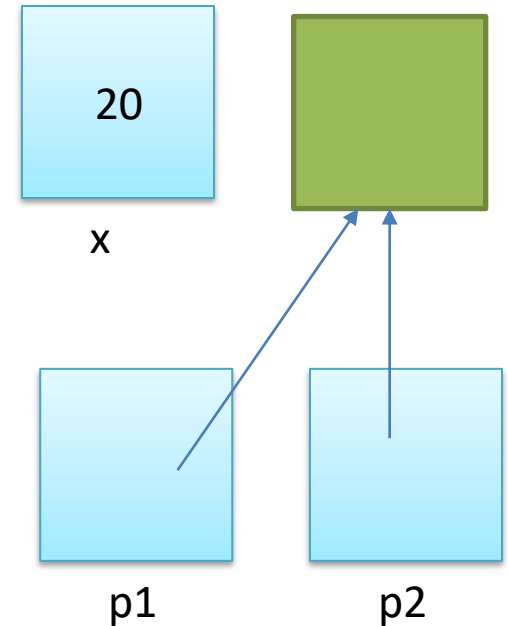
Another Example Step by Step

- `int x;`
- `int *p1;`
- `int *p2 = (int *) malloc(sizeof(int));`
- `p1 = &x;`
- `*p1 = 20;`



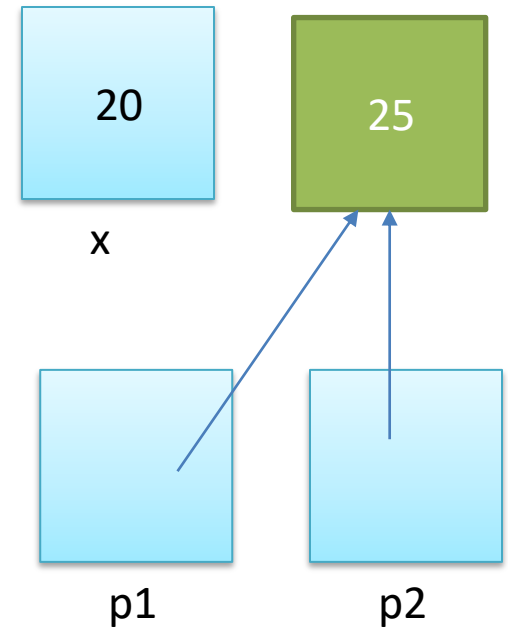
Another Example Step by Step

- `int x;`
- `int *p1;`
- `int *p2 = (int *) malloc(sizeof(int));`
- `p1 = &x;`
- `*p1 = 20;`
- `p1 = p2;`



Another Example Step by Step

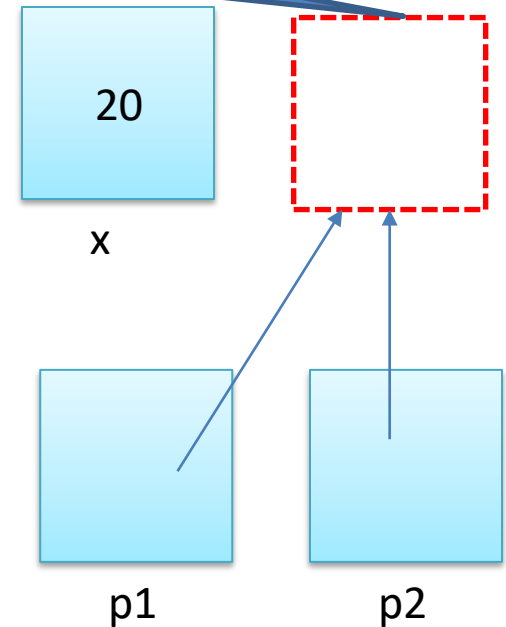
- `int x;`
- `int *p1;`
- `int *p2 = (int *) malloc(sizeof(int));`
- `p1 = &x;`
- `*p1 = 20;`
- `p1 = p2;`
- `*p2 = x + 5;`



Another Example Step by Step

Memory
Released!

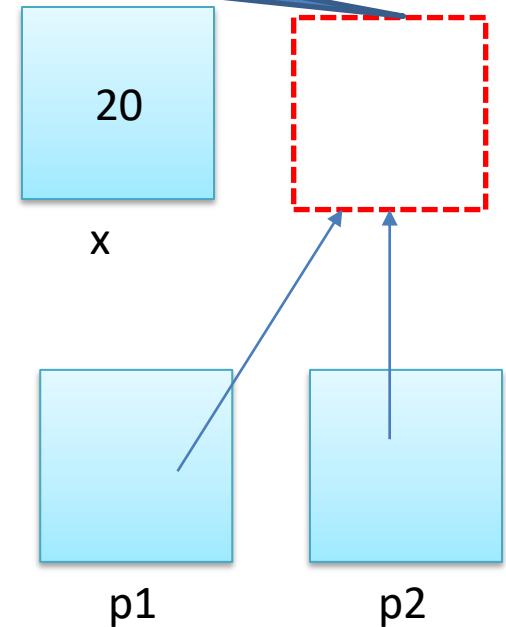
- `int x;`
- `int *p1;`
- `int *p2 = (int *) malloc(sizeof(int));`
- `p1 = &x;`
- `*p1 = 20;`
- `p1 = p2;`
- `*p2 = x + 5;`
- `free(p2);`



Another Example Step by Step

Memory
Released!

- `int x;`
- `int *p1;`
- `int *p2 = (int *) malloc(sizeof(int));`
- `p1 = &x;`
- `*p1 = 20;`
- `p1 = p2;`
- `*p2 = x + 5;`
- `free(p2);`
- `*p1 = 100; //What do you expect at this line?`



You do not have permission to that
address! **Generally, Segmentation
fault**

realloc

- There might be cases when the allocated array size is not enough and you will need to resize the array.
- How would you approach this?
- Naïve approach would be:
 - Allocate new memory
 - Copy the old data to the new allocated array
 - Free the old array.
- We can avoid extra work through `realloc` function.
- `void *realloc(void *ptr, size_t size);`
- So here, `ptr` is the old pointer, and `size` is the new size.
- It will allocate `size` amount of bytes and copy the content from the allocated data in `ptr` and return void pointer.
- Let's say `values` is an integer pointer and already allocated to `numVals` size. The following line will reallocate.
- `values = (int*)realloc(values,(numVals+EXTRA)*sizeof(int));`
- See uploaded example.

Example of realloc

```
#include <stdio.h>
#include <time.h>
#define EXTRA 10
int main() {
    int numVals;
    srand(time(0));
    printf("How many numbers?\n");
    scanf("%d", &numVals);

    int* values =
(int*)malloc(numVals*sizeof(int));
    int i;
    for (i=0; i<numVals; i++)
        values[i] = rand()%100;
    for (i=0; i<numVals; i++)
        printf("%d ", values[i]);
    printf("\n");

    //see the right side for remaining
    lines of the code
```

```
values =
(int*)realloc(values, (numVals+EXTRA) *
sizeof(int));

    for (i=0; i<EXTRA; i++)
        values[i+numVals] =
            rand()%100;

    numVals += EXTRA;

    for (i=0; i<numVals; i++)
        printf("%d ", values[i]);

    printf("\n");

    free(values);
    return 0;
}
```

```
//output
How many numbers?
5
97 62 7 74 48
97 62 7 74 48 44 23 75 61 64 69 92 39 23 58
Process returned 0 (0x0)  execution time : 3.112 s
```

So, what we have covered in our last lecture?

i Forgot



i 4got



Now, as we know that dynamically allocated memory is not destroyed automatically, we can return a dynamically allocated array from function!

Dynamically allocating array in function

- We will write a function that will allocate memory for an array, fill-up the array and return the allocated memory!
- So, in this way, we can actually return a dynamically allocated array
- Why is it possible now?
 - Because we know that dynamically allocated memories are alive until you free them!
- We can free the memory from main function!
- See the Example:

```
int* readArray(int size) {  
    int* p = (int *)malloc(size*sizeof(int));  
    for (int i = 0; i<size; i++)  
        scanf("%d", &p[i]);  
    return p;  
}
```

1. Allocate memory for size number of int and p is pointing to the beginning of the memory
2. Fill-up the array
3. Return the first address. As the memory is dynamically allocated, they will be alive and can be access by the main function

How to call it from the main function?

```
int main()  
{  
    int size;  
    printf("How many numbers?:");  
    scanf("%d", &size);  
    int* numbers = readArray(size);  
    //now you can use numbers like an array of size size  
    //after performing all the operation on numbers, free it  
    free(numbers);  
}
```

p



After allocating memory, you can use p like an int array, like p[0], p[1]..

Calling the function and keeping the pointer returned by the function

Free the memory at the end

An Example Code

```
int* readArray(int size) {
    int* p = (int *)malloc(size*sizeof(int));
    for (int i = 0; i<size; i++)
        scanf("%d", &p[i]);
    return p;
}

int findMax(int arr[], int len) {
    int i, max = arr[0];
    for(i = 1; i<len; i++)
        if(arr[i]>max)
            max = arr[i];

    return max;
}

int main(void) {
    int size;
    int *myarr;
    printf("How many numbers? ");
    scanf("%d", &size);
    myarr = readArray(size);

    printf("Printing the array from main function: ");
    for(int i=0; i<size; i++)
        printf("%d ", myarr[i]);

    printf("\nThe max number in the array is %d", findMax(myarr,size));

    free(myarr); //free the memory

    return 0;
}
```

How to create a dynamically allocated struct from a function

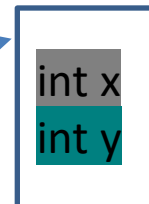
How to create a dynamically allocated array of structs from a function

- Consider the following structure that can be used to store a point (x,y)

```
struct point {  
    int x;  
    int y;  
};
```

- We want to write a function that will create a point and initialize it based on user input

temp



```
struct point* createPoint(){  
    struct point * temp;  
    temp = (struct point*)malloc(sizeof(struct point));  
    scanf("%d %d", &(temp->x), &(temp->y));  
    return temp;  
}
```

A complete example of creating point in function

```
#include <stdio.h>
#include <stdlib.h>

typedef struct point{
    int x;
    int y;
}point;

point* createPoint(){
    point * temp;
    temp = (point*)malloc(sizeof(struct point));
    scanf("%d %d", &(temp->x), &(temp->y));
    return temp;
}

int main(void) {
    point *p;
    p = createPoint();

    printf("point in main %d %d", p->x, p->y);

    free(p);
    return 0;
}
```

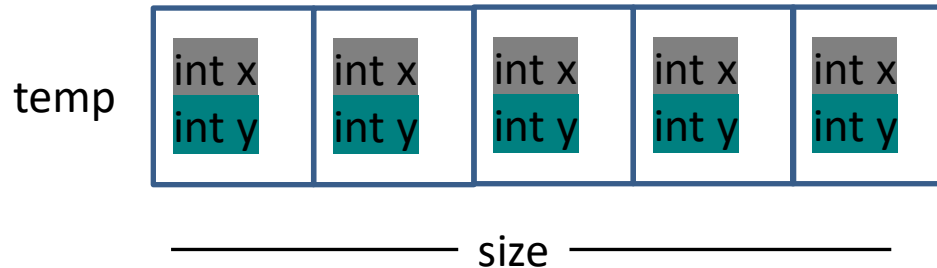
How to create a dynamically allocated array of structs from a function

Creating an array of point

- Very similar to allocating an int array dynamically. Only difference is it is struct type instead of int.
- Example structure:

```
struct point {  
    int x;  
    int y;  
};
```

- We want to create something like this, where the size is dynamic:



- So, how can we do that?
 - `struct point* temp;`
 - `temp = (struct point*)malloc(size*sizeof(struct point));`

How to create a dynamically allocated array of structs from a function

- Very similar to allocating an int array dynamically. Only difference is it is struct type instead of int.

- Example structure:

```
struct point {  
    int x;  
    int y;  
};
```

- The following function takes array size and max value of x and y to be generated. Then it dynamically allocate array point filled with random values. Then returns a pointer that points the front of the array.

```
struct point* createRandPoints(int size, int maxVal) {  
    struct point* temp;  
    temp = (struct point*)malloc(size*sizeof(struct point));  
    int i;  
    for (i=0; i<size; i++) {  
        temp[i].x = 1 + rand()%maxVal;  
        temp[i].y = 1 + rand()%maxVal;  
    }  
    return temp;  
}
```

#Once the space is allocated, we treat each array location as an individual struct, using . to access its components.

#To free this array, if we had a pointer my_pts pointing to the array, do the following: **free(my_pts);**

```

struct point{
    int x;
    int y;
};

struct point* createRandPoints(int size, int maxVal) {
    struct point* temp;
    temp = (struct point*)malloc(size*sizeof(struct point));
    int i;
    for (i=0; i<size; i++) {
        temp[i].x = 1 + rand()%maxVal;
        temp[i].y = 1 + rand()%maxVal;
    }
    return temp;
}

int main(void) {
    struct point *p;
    int size;
    printf("How many points? ");
    scanf("%d", &size);

    p = createRandPoints(size, 100);

    printf("Printing the random points: \n");
    for(int i=0; i<size; i++)
    {
        printf("point %d, x = %d, y = %d\n", i, p[i].x, p[i].y);
    }

    free(p);

    return 0;
}

```

**Dynamically allocating a structure
from a function that has a
dynamically allocated array in it**

Dynamically allocated a structure from a function that has a dynamically allocated array in it

- We will create the following struct and return a pointer to it from a function:

```
struct BigInteger {  
    int* digits;  
    int size;  
};
```

- The purpose of the structure is to store a very big integer with size number of digits
- To store that we will use digits as an array
 - Each array element will be a digit of the number
 - So, the entire array will represent big integer with size digits
- We will write a function that will allocate memory for a BigInteger
 - Then within the BigInteger, we will allocate memory for digits, which will be an int array of size *size*.
 - Then we will randomly fillup the array with random digits
 - We want the right most digit to be non-zero

Dynamically allocated a structure from a function that has a dynamically allocated array in it

- We will create the following struct and return a pointer to it from a function:

```
struct BigInteger {  
    int* digits;  
    int size;  
};
```

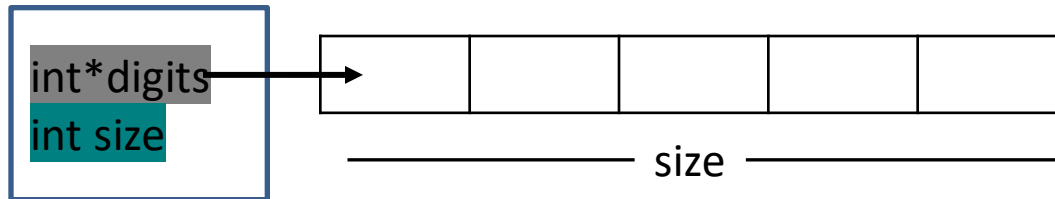
- Now we want to dynamically allocate memory for an integer structure. How can we do that?

```
- struct BigInteger* temp;  
temp = (struct BigInteger *) malloc(sizeof(struct BigInteger));
```

- Next, we want to dynamically allocate memory for the digits which will be an array of int with size number of elements. How can we do that?

```
- temp->digits = (int *) malloc(numDigits*sizeof(int));
```

- The memory allocation picture will be like this:



- Now, if we want to do it from a function, what should you return?
- How the function signature should look like?

Dynamically allocated structure from a function

- We will create the following struct and return a pointer to it from a function:

```
struct BigInteger {  
    int* digits;  
    int size;  
};
```

- The following function creates a random struct integer dynamically and returns a pointer to it:

```
struct BigInteger* createRandBigInt(int numDigits) {  
    struct integer* temp;  
    temp = (struct BigInteger *) malloc(sizeof(struct BigInteger));  
    temp->digits = (int *) malloc(numDigits*sizeof(int));  
    temp->size = numDigits;  
    temp->digits[numDigits-1] = 1 + rand()%9; //we just want last digit to be nonzero  
    int i;  
    for (i=0; i<numDigits-1; i++)  
        temp->digits[i] = rand()%10;  
    return temp;  
}
```

- So, let's see the code: How many mallocs in the above code? What are their purpose?
- First one allocated space for the struct itself. This space is just for one integer pointer (small amount space) and one integer.
- The second malloc allocates space for the array (this is potentially a large amount of space).

How would you free the memory for the example

- To properly free the memory from this whole structure,
- imagine we had a variable **p** of type **struct BigInteger*** pointing to a **struct BigInteger**.
- These two lines would be necessary to free all the memory for the structure:

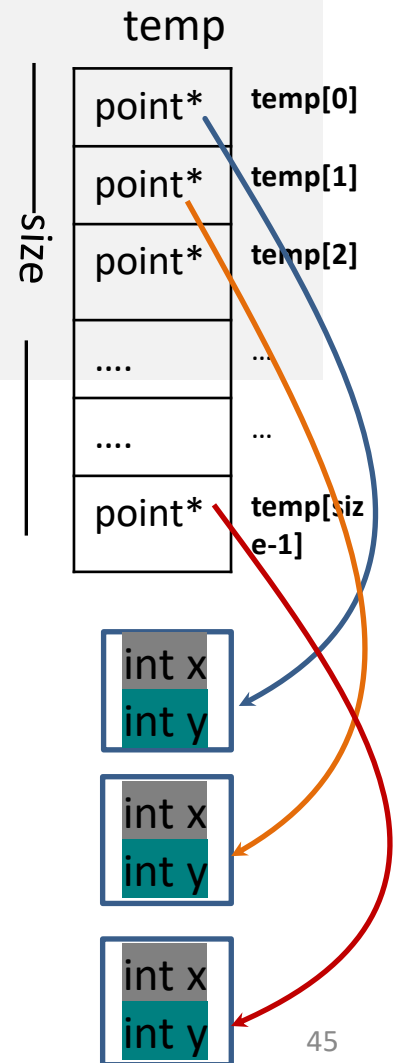
```
free (p->digits) ;  
free (p) ;
```

Creating a dynamically allocated **array** of **pointers to structs**

Creating a dynamically allocated **array of pointers to structs**

- It would be very similar to the previous example, but this time, our array elements will only store a **POINTER** to the struct instead of the struct itself.
- So, our picture will be like the following picture.
- So, what type of array is it?
 - What is the size of that array?
 - What type of variable is the temp?
 - What can we store inside the array element?
 - Can we use each of them for dynamic memory allocation?

- So, temp is an array of structure pointer.
 - It means, each element of the array will be a pointer
 - So, making temp as a simple structure pointer would not have the power to hold address of another pointer
 - It means temp has to be a a pointer of pointer
 - The equivalent lines of code for the following picture is this:
 - `struct point** temp;`
 - `temp = (struct point**)malloc(size*sizeof(struct point*));`
- Now, we have an array of point pointers and we can use each of them to allocate memory for many points as we wish.
- It means, each temp[i] can be another array!
- But in our example, we will just allocate one structure for each of them
- `temp[i] = (struct point*)malloc(sizeof(struct point));`



How to create a dynamically allocated **array of pointers to structs**

- So, the function will look like this

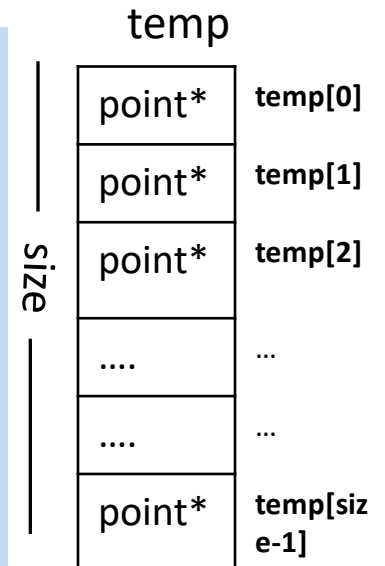
```
struct point** createRandPoints(int size, int maxVal) {  
    struct point** temp;  
    temp = (struct point**)malloc(size*sizeof(struct point*));  
    int i;  
    for (i=0; i<size; i++) {  
        temp[i] = (struct point*)malloc(sizeof(struct point));  
        temp[i]->x = 1 + rand()%maxVal;  
        temp[i]->y = 1 + rand()%maxVal;  
    }  
    return temp;  
}
```

- **First**, notice the double pointer – **the first is for the array, the second is for the contents of each array element**. (Note: This same declaration could be used for a 2-D array...). Our first allocation is for the array. From there, for each array element, we must allocate space for each individual struct. **Finally**, notice the use of the -> since temp[i] is a pointer.

How would you free the memory for the last example?

- We must free everything in the same fashion (each element first, then the array). Here is some code to show this process:

```
struct point** my_pts = createRandPoints(100,  
1000);  
// Do something with my_pts.  
// Frees each individual point pointer.  
int i;  
for (i=0; i<100; i++)  
    free(my_pts[i]);  
// Frees the memory that stores the main array.  
free(my_pts);
```



Can you modify the previous code to create several dynamically allocated array of points? (2D array of points!)

How to dynamically allocate a two-dimensional array

How to dynamically allocate a two-dimensional array

- Here is some code that allocates a two dimensional array of integers (or rather, an array of an array of integers) with n rows and m columns. Assume that n and m are integer variables that have already been given meaningful values.

```
int** array = (int**)malloc(sizeof(int*) *n) ;  
int i;  
for (i=0; i<n; i++)  
    array[i] = (int*)malloc(sizeof(int) *m) ;
```

- Notice that the first cast is to int** and second set of casts are to int*.
- These are the types of array and array[i], respectively.

Also note that we need `sizeof(int*)` in the first malloc, since array will be an array of int*, but we need `sizeof(int)` for the second set of mallocs, since each array[i] will be an array of integers.

Now, here is how we free this memory:

```
for (i=0; i<n; i++)  
    free(array[i]) ;  
free(array) ;
```

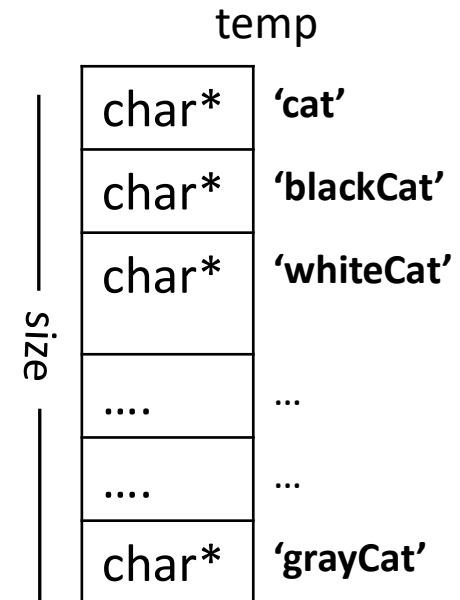
Dynamic memory allocation for an array of Strings

Dynamic memory allocation for an array of Strings

- Consider you have a file with words.
- The first line says how many words will be there and the next lines contain the words.
- Example:

```
6
cat
blackCat
whiteCat
yellowCat
orangeCat
grayCat
```

- Now you want to write a function to read the strings and print them.
- We want to use dynamic memory allocation for storing the array of string and also for storing the string by dynamically allocating memory.



Dynamic memory allocation for an array of Strings

- Let us see our main function:
- Why we need double pointer?
- Because we want to store an array of string, where:
 - The size of the array is dynamic
 - Each element in the array is a string which will be dynamically allocated
- What are we passing to readList function?
 - Can you write the signature of the function?
 - See, we are passing &numWords to the function
 - Why we do that?
 - Because we want it to get updated in the function (pass by reference!)

```
int main() {  
  
    char fileName[50];  
    char** pList;  
    int numWords;  
  
    // Get the filename.  
    printf("Enter the file name.\n");  
    scanf("%s", fileName);  
  
    // Read in the list storing it in pList.  
    // numWords will store the number of  
    // words in the file.  
    pList = readList(fileName, &numWords);  
  
    // Print these words to check that our  
    // read worked and free the memory.  
    printList(pList, numWords);  
    freeList(pList, numWords);  
    return 0;  
}
```

Dynamic memory allocation for an array of Strings

- Why we don't have & here?
- Why we have * here?
- As WordsCount itself a pointer, which actually changing everything to numWords variable in main function.
- Why are we adding 1?
- So, this code reads each string from the file and allocate memory for that string into our array and store it like the picture in the last slide,

```
char** readList(char* fileName, int* wordsCount) {  
    char** pList = NULL;  
    char word[MAX_STRING_LENGTH];  
    int i, wordLength;  
  
    FILE* pFile = fopen(fileName, "r"); // open file.  
  
    if (pFile != NULL){  
        /* read the 1st line, get to know how many words in the dictionary. */  
        fscanf(pFile, "%d", wordsCount);  
        /* Here we allocate space a pointer for each word in the list. Note that the  
        space for the words themselves is NOT allocated here. */  
        pList = malloc(*wordsCount * sizeof(char *));  
  
        for (i = 0; i < *wordsCount; i++){  
            // read in this word.  
            fscanf(pFile, "%s", word);  
            // Allocate one extra spot for the null character.  
            wordLength = strlen(word) + 1;  
            // Allocate space for this individual word.  
            pList[i] = malloc(wordLength*sizeof(char));  
            // copy the word to the memory block.  
            strcpy(pList[i], word);  
        }  
        fclose(pFile); // close file.  
    }  
    return pList;  
}
```

- The full code is available on webcourses.

Other examples are discussed in the
lecture

Reference

- Arup's note on dynamic memory allocation:
<http://www.cs.ucf.edu/~dmarino/ucf/transparency/cop3502/lec/DynMemAlloc.pdf>